核心：`throws` 是写给编译器看的“提前报备”

先分开两个概念：

1. `throw new XXXException()`：运行时动作 → 真的把异常扔出来，让程序中断。
2. `throws 异常类`：写在方法签名上，**只是给编译器的提示文字，不是抛出动作。**

1. `try` 块里面代码抛出异常 → 进入匹配的那个 `catch`
2. 在这个 `catch` 内部 `throw` 新异常：
   - 已经**离开 try 的保护范围了**，后面并列的这些 catch 只管 try 里面发生的异常
   - 并列的 catch 互相之间不能抓对方里面抛出的异常
3. 这个新异常会**往外跑**，交给**外层的 try-catch**（调用这一段代码的地方）去捕获

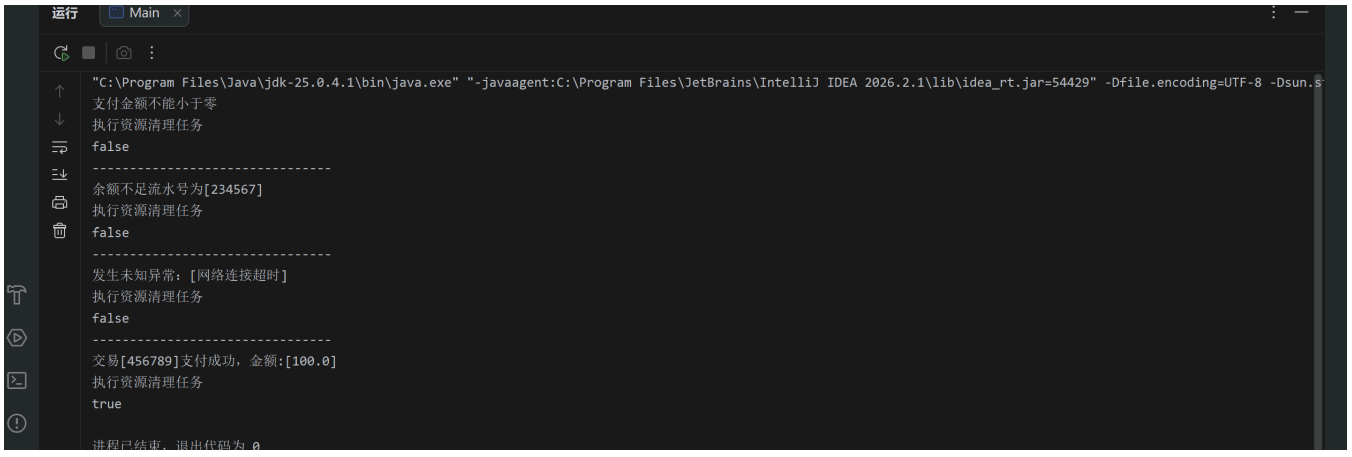
执行到 `throw` 的瞬间：

✅ throw 之前的代码跑完

❌ **throw 之后的代码直接放弃，catch 块提前终止，不会跑完剩下的**

异常向外抛出。

运行结果：



代码：

```
package Bao5;

public class Main {
    public static void main(String[] args) {
        Service service = new Service();
        System.out.println(service.test(-1, "123456"));
        System.out.println("--------------------------------");
        System.out.println(service.test(11222, "234567"));
        System.out.println("--------------------------------");
        System.out.println(service.test(500, "345678"));
        System.out.println("--------------------------------");
```

```
        System.out.println(service.test(100, "456789"));

    }

}


package Bao5;

public class Service {
    public boolean test(double pay, String transactionId) throws SystemMaintenanceException{
        //创建对象
        PaymentProcessor paymentProcessor = new PaymentProcessor();
        try{
        //调用方法
            paymentProcessor.paymentWays(pay, transactionId);
        //如果没有异常，才会执行这行代码
            return true;
        }
        //捕获异常，并打印信息
        catch(InsufficientFundsException e){
            System.out.println(e.getMessage()+"流水号为["+e.getTransactionId()+"]");
            return false;
        }
        //捕获异常，并抛出新异常
        catch(IllegalArgumentException e1){
            try {
                throw new SystemMaintenanceException(e1.getMessage(), e1);
            }
            catch(SystemMaintenanceException e2){
                System.out.println(e2.getMessage());
            }
            return false;
        }
        //捕获异常，并打印信息
        catch(Exception e){
            System.out.println("发生未知异常：["+e.getMessage()+"]");
            return false;
        }finally{
            System.out.println("执行资源清理任务");
        }

    }
}


package Bao5;

public class PaymentProcessor {
    public void paymentWays(double pay, String transactionId) throws
IllegalArgumentException, InsufficientFundsException {
        if(pay < 0){
            throw new IllegalArgumentException("支付金额不能小于零");
```

```
        }else if(pay>10000){
            throw new InsufficientFundsException("余额不足", transactionId);
        }else if(pay==500){
            throw new SystemMaintenanceException("网络连接超时", new NullPointerException());
        }else{
            System.out.println("交易["+transactionId+"]支付成功，金额:["+pay+"]");
        }
    }
}


package Bao5;

public class InsufficientFundsException extends Exception {
    private String transactionId;

    public InsufficientFundsException(String message, String transactionId) {
        super(message);
        this.transactionId = transactionId;
    }

    public String getTransactionId() {
        return transactionId;
    }
}


package Bao5;



public class SystemMaintenanceException extends RuntimeException {
//构造方法，用于异常链
    public SystemMaintenanceException(String message, Throwable cause) {
        super(message, cause);
    }
}
```